OTPFloodGuard: A Public-Evidence-Constrained Benchmark for Lightweight OTP Flooding Detection

Wenche An[1], Kamran Aziz[2,*]
[1]Department of Computer Science,
Hainan Bielefeld University of Applied Sciences,
Hainan, China
[2]Department of Digital Technologies,
Hainan Bielefeld University of Applied Sciences,
Hainan, China
wenche.an.24@stu.hainan-biuh.edu.cn; kamran.aziz@hibiuh.edu.cn
*Corresponding author: Kamran Aziz.

***Abstract*—One-time password (OTP) services can be abused through OTP flooding and SMS pumping, but public, incident-labeled OTP request-window datasets remain limited because such logs may contain sensitive authentication metadata. This paper presents OTPFloodGuard, a public-evidence-constrained simulated benchmark whose construction procedure maps public OTP/SMS abuse evidence to threat assumptions, window-level features, difficulty controls, and evaluation diagnostics. As a baseline use case, Random Forest achieves 0.9617 F1 on the Overlap benchmark, 0.9514 $\pm$ 0.0052 mean F1 across seven stratified splits, and 0.9496 $\pm$ 0.0057 mean F1 across five separately regenerated benchmark instances. Performance decreases from 0.9876 F1 on Easy to 0.8967 on Adaptive, and generator-shift testing reduces Random Forest F1 to 0.8806. These results position OTPFloodGuard as an early-warning risk-scoring benchmark under simulated assumptions, not as a proxy for production accuracy.**

***Index Terms*—OTP flooding, SMS pumping, public-evidence-constrained benchmark, simulated security data, lightweight machine learning**

I. Introduction

OTP verification is widely used for signup, login, password reset, and transaction confirmation, but OTP workflows can themselves become abuse targets. OTP flooding, SMS pumping, and Artificially Inflated Traffic (AIT) have been documented in fraud-prevention guidance, threat-intelligence sources, and telecom-security research [1]–[8]. Attackers repeatedly trigger OTP messages through automated signups, abused verification endpoints, or scripted request flows, increasing messaging costs, exhausting verification capacity, denying service to legitimate users, or supporting broader identity-abuse campaigns [1], [3]–[8]. Large-scale AIT work further shows that attackers may distribute SMS authentication requests across accounts, phone numbers, or devices to bypass naive rate limits [8].

The main challenge is data availability. Public, incident-labeled OTP request-window datasets remain limited because such logs may reveal sensitive authentication metadata, phone-number patterns, IP/device information, carrier routing, and incident labels. Unrelated public datasets are poor substitutes: intrusion datasets capture packet/flow behavior [9], SMS spam datasets classify content [10], and credit-card fraud datasets model payment transactions [11]. OTPFloodGuard therefore frames the absence of public OTP flooding logs as a benchmark-design problem: how to construct, stress-test, and report OTP abuse simulations responsibly.

This paper addresses four research questions:

RQ1: Can lightweight models outperform simple rule-based baselines under public-evidence-constrained simulated OTP flooding scenarios?

RQ2: How does detection performance change across Easy, Overlap, and Adaptive benchmark settings?

RQ3: Which feature groups contribute most to detection performance, and what patterns characterize model errors?

RQ4: How do attack intensity and threshold selection affect early-warning risk-scoring and conservative-response use cases?

The contributions are:

1) A public-evidence-constrained benchmark construction method that maps OTP/SMS abuse evidence into explicit threat assumptions, feature definitions, and replaceable simulation controls.
2) A multi-difficulty OTP flooding benchmark with Easy, Overlap, and Adaptive regimes, designed to avoid evaluating only trivially separable synthetic attacks.
3) A stress-tested lightweight baseline evaluation that combines multi-split stability, generator-seed sensitivity, generator-shift robustness, feature-group ablation, threshold sensitivity, and error-case inspection to evaluate benchmark behavior beyond single-score reporting.

II. Background and Related Work

OTP flooding and SMS pumping are authentication-abuse and telecom-fraud patterns involving repeated OTP requests, costly destination routing, or overloaded verification services. Public guidance identifies request velocity, verification failure, destination concentration, carrier/country anomalies, and repeated source behavior as operational signals [1]–[6]. Peer-reviewed telecom-fraud and AIT work supports engineered behavioral indicators and shows that artificial SMS traffic can be distributed across accounts, phone numbers, or devices to evade naive rate limits [7], [8]. OTPFloodGuard does not solve

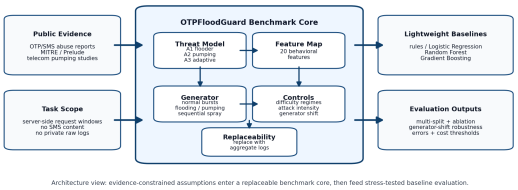

Fig. 1: OTPFloodGuard benchmark architecture.

the same real-log detection problem; it uses this evidence to motivate benchmark features and the need for reusable OTP flooding benchmark frameworks.

Rule-based controls are simple and auditable, but fixed thresholds can fail when legitimate events resemble attacks or attackers lower their rate, distribute infrastructure, or avoid obvious failure patterns. Lightweight tabular models such as Logistic Regression, Random Forest, and Gradient Boosting are appropriate baselines because they combine weak signals without heavy deep learning infrastructure [12]–[14]. Random Forest impurity-based feature importance is used only as a training-set ranking heuristic, while permutation importance is a separate diagnostic [15]. Methodological work on closed-world security evaluation and dataset development motivates explicit assumptions, task alignment, and cautious interpretation [19]–[21].

III. Threat Model and Scope

The benchmark studies server-side OTP request-window behavior under simulated flooding, pumping, and sequential spray patterns. The defender observes aggregate metadata such as request counts, success/failure rates, repeated IP/device behavior, destination distribution, and timing, but does not inspect SMS content or raw phone numbers. The attacker profiles are A1 Naive Flooder, A2 Pumping-Oriented Attacker, and A3 Adaptive Low-Rate Attacker. The paper does not address SIM swap, local malware, phishing, man-in-the-middle attacks, SMS content analysis, carrier billing-cost modeling, or production automatic-blocking policies; local-device SMS OTP threats are outside this request-window benchmark [16].

IV. Benchmark Design

A. Public Evidence and Design Assumptions

Table I summarizes evidence-supported risk signals and explicit benchmark design assumptions. The cited sources motivate the modeled abuse indicators, while the final two rows define controlled stress conditions for hard normal cases and adaptive attack cases. These choices make the benchmark assumptions visible and replaceable, but they do not establish production distributions or real-world model validity.

B. Pipeline

Fig. 1 presents the OTPFloodGuard benchmark architecture.

The generator pipeline maps public evidence to threat assumptions, attacker profiles, feature-level transformations, difficulty controls, synthetic OTP request windows, and evaluation diagnostics. The generator does not reconstruct private OTP traffic. Instead, it creates controlled benchmark windows under transparent assumptions so that each result can be traced to a threat assumption, feature definition, and generator control.

TABLE I: Public evidence and benchmark design assumptions.

Evidence or benchmark assumption	Threat assumption	Simulated behavior and features
Twilio and RingCaptcha describe abnormal OTP/SMS request bursts and resource exhaustion [1], [4]	Automation increases short-window activity	Increase request count and velocity; otp_requests, request_velocity_per_sec, avg_interarrival_ms
Twilio Verify guidance discusses OTP abuse signals and fraud controls [2]	Abuse traffic often has lower verification completion	Increase failure rate and reduce success rate; failure_rate, success_rate
IPQualityScore, MITRE, and Prelude describe toll-fraud, SMS pumping, and destination or routing concentration [3], [5], [6]	Pumping may concentrate prefixes, carriers, countries, or destinations	Increase prefix concentration or alter country/carrier distribution; prefix_concentration, country_entropy, carrier_entropy, risk_country_ratio
Public fraud guidance and telecom traffic-pumping research identify behavioral abuse indicators [1]–[7]	Source behavior may reveal simple or adaptive automation	Vary IP/device reuse and ratios; unique_ip_count, unique_device_count, repeat_ip_ratio, ip_phone_ratio, device_phone_ratio
Benchmark stress assumption: legitimate bursts and delivery-failure-like behavior may resemble abuse signals	Some normal windows are intentionally made difficult	Inject hard normal windows with high volume or failure-like behavior; otp_requests, failure_rate, prefix_concentration
Adaptive stress assumption: attack features are shifted toward normal distributions to test threshold evasion	Low-rate attacks are designed to resemble normal traffic	Move attack windows toward normal feature distributions across the modeled behavioral features

C. Window Definition and Behavior Classes

Each sample is a short OTP monitoring window. The primary setting uses 60-second windows. The class label is binary: normal or attack. Attack windows include OTP flooding, SMS pumping, and sequential phone-number spray. Normal windows include ordinary traffic and harder normal cases such as legitimate registration bursts or delivery-failure-like windows.

The 60-second window is a benchmark design choice rather than a production-validated aggregation interval. Evaluating alternative window lengths requires event-level request sequences and is left for future validation.

D. Feature Groups

The benchmark implements 20 window-level numeric features grouped into five roles: window/volume, destination behavior, source/device behavior, verification outcome, and context/routing. Exact definitions and summary statistics are exported in `feature_dictionary.csv`. Some features are correlated by design; for example, success_rate and failure_rate are complements. The evaluation therefore uses feature-group ablation and permutation importance rather than treating impurity-based feature importance as a causal explanation.

E. Difficulty Levels

The benchmark uses three difficulty regimes. Easy makes attacks clearer through higher velocity, failure, reuse, or

destination concentration. Overlap is the main benchmark and includes legitimate bursts, delivery failures, and low-intensity attacks. Adaptive is the stress-test regime: attack windows are shifted toward normal-window distributions by reducing velocity, distributing IP/device identifiers, and imitating more normal success behavior.

The implementation-level controls are explicit. Easy multiplies attack request counts by 1.35, attack success rates by 0.55, attack prefix concentration by 1.18, and attack repeat-IP ratio by 1.12. Adaptive blends count, velocity, destination, context, IP/device, and success/failure features toward sampled normal windows using a 0.22 attack signal + 0.78 normal signal blend. These are controlled stress-test parameters, not production estimates of attack prevalence or fraud rates; they define perturbation strengths for sensitivity and robustness testing. The attack-intensity experiment in the Experimental Setup varies a separate stress-test parameter, alpha.

F. Generation and Reproducibility

The primary Overlap benchmark is generated with generator seed 42. The primary Overlap dataset contains 12,000 windows: 6,986 normal windows and 5,014 attack windows. Attack windows include 2,187 flooding, 1,882 SMS pumping, and 945 sequential spray windows. Easy and Adaptive use the same sample count and class distribution as Overlap; only simulation controls change. The generator samples a behavior class, creates count-based and behavioral features, injects legitimate-burst and low-intensity cases, recalculates derived ratios, and exports data, metrics, plots, and error-analysis files.

The code and generated artifacts are available in a public repository.[1] The repository contains the simulation script, generated CSV results, paper figures, and commands for quick verification and full reproduction. All included data are simulated; no real OTP logs or private user records are included.

G. Benchmark Scope and Replaceability

OTPFloodGuard is intended for research benchmarking, controlled model comparison, early-warning risk-scoring evaluation, and analyst-facing studies. It is not intended for production automatic-blocking policy, real-world accuracy claims, carrier billing estimation, or replacement for incident-labeled logs. Its replaceable components include feature distributions, difficulty controls, attack ratios, country/carrier assumptions, and threshold policies; future versions should replace these with anonymized aggregate OTP request statistics or labeled private incident windows when available.

V. Experimental Setup

The evaluated methods include Logistic Regression, Random Forest, Gradient Boosting, a tuned four-signal rule baseline, and a velocity-and-failure rule baseline. All use the same stratified 80/20 split. Model fitting, feature ranking, scaling, rule-threshold selection, and cost-sensitive threshold selection use only the training portion; the held-out test set is reserved for final reporting. The main results use generator seed 42 and split seed 42, while seven split seeds and five generator seeds measure partition and generator sensitivity.

[1]https://github.com/Anwenche/otpfloodguard-benchmark

TABLE II: Main results on the Overlap benchmark.

Model	Feature set	Precision	Recall	F1	ROC-AUC	PR-AUC
Random Forest	Full	0.9469	0.9771	0.9617	0.9942	0.9913
Gradient Boosting	Full	0.9545	0.9611	0.9578	0.9942	0.9917
Logistic Regression	Full	0.9100	0.9272	0.9185	0.9820	0.9751
Velocity + Failure Rule	Rules	0.9190	0.8824	0.9003	N/A	N/A
Tuned Rule Baseline	Rules	0.9073	0.8883	0.8977	N/A	N/A

Model configurations are fixed before held-out evaluation: Random Forest uses 250 trees, max depth 10, minimum leaf size 3, and no class weighting; Logistic Regression uses a training-fitted StandardScaler and L2 regularization; Gradient Boosting uses 100 estimators, learning rate 0.1, depth 3, full subsampling, and log loss. Full parameters, rule grids, feature-ranking metadata, and software versions are exported in the repository. Random Forest is the primary diagnostic model for threshold, feature, and error analyses, but this designation is fixed before test evaluation and is not selected on held-out performance.

The Top-15, Top-10, and Top-5 subsets use training-set Random Forest impurity-based importance; cross-validated permutation importance is reported only as an interpretability diagnostic. Metrics are precision, recall, F1-score, ROC-AUC, and PR-AUC, with PR-AUC included because precision-recall analysis is useful when false alerts matter operationally [18]. A generator-shift evaluation transforms held-out rows with a separately seeded, class-conditional, label-preserving generator; no model, scaler, selected feature set, model-score threshold, or rule threshold is refitted or retuned. Model scores are empirical decision scores, not calibrated attack probabilities.

Attack intensity is implemented as a scaling factor applied to attack windows relative to sampled normal windows:

$$x'_{i,f} = [n_{i,f} + \alpha\,(x_{i,f} - n_{i,f})]\,\epsilon_{i,f}, \quad \epsilon_{i,f} \sim \mathcal{N}(1, 0.04^2) \tag{1}$$

where $x_{i,f}$ is the original attack value for sample i and feature f, $n_{i,f}$ is a sampled normal reference, $\alpha \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$, and $\epsilon_{i,f}$ is per-sample, per-feature multiplicative noise drawn from $\mathcal{N}(1, 0.04^2)$. Ratio-valued features are clipped to predefined valid ranges. Count-valued features are rounded to the nearest integer and clipped to feature-specific lower bounds.

VI. Results

A. Main Model Comparison

Table II reports the main results on the Overlap benchmark.

For RQ1, Random Forest and Gradient Boosting achieve higher reported F1 and recall than the rule baselines under the Overlap simulated benchmark settings. This result does not indicate deployment readiness; it suggests that combining multiple window-level signals may improve over simple threshold rules under the simulated assumptions.

The benchmark contains 12,000 windows, including 5,014 attack windows and 6,986 normal windows. This 41.8% attack proportion is a controlled benchmark base rate chosen to

TABLE III: Difficulty-regime results.

Difficulty	Random Forest F1	Random Forest recall	Tuned-rule F1	Tuned-rule recall
Easy	0.9876	0.9920	0.9447	0.9621
Overlap	0.9617	0.9771	0.8977	0.8883
Adaptive	0.8967	0.8824	0.7236	0.7438

TABLE IV: Generator-shift robustness results.

Model	Precision	Recall	F1	ROC-AUC	PR-AUC
Gradient Boosting	0.9261	0.8624	0.8931	0.9722	0.9578
Random Forest	0.8941	0.8674	0.8806	0.9672	0.9490
Logistic Regression	0.8782	0.8624	0.8702	0.9476	0.9382
Tuned Rule Baseline	0.8392	0.6610	0.7395	N/A	N/A
Velocity + Failure Rule	0.8473	0.6361	0.7267	N/A	N/A

support model comparison, error analysis, and stress testing, not an estimate of production OTP abuse prevalence. Low production prevalence can substantially reduce precision even when benchmark F1 is high, so this section reports a prevalence-sensitivity calculation and exports the projections in `prevalence_sensitivity.csv`. The generated dataset size and class balance are exported in `benchmark_config.json`.

Across seven stratified train-test partitions of the same generated Overlap dataset, Random Forest obtains 0.9514 $\pm$ 0.0052 mean F1 and 0.9748 $\pm$ 0.0059 mean recall; Gradient Boosting obtains 0.9463 $\pm$ 0.0067 mean F1 and 0.9570 $\pm$ 0.0064 mean recall. The two rule baselines remain near 0.89 F1. These results measure sensitivity to data partitioning within one generated dataset and are not confidence intervals from independent dataset replications. Generator-seed sensitivity across five separately regenerated Overlap benchmark instances gives Random Forest 0.9496 $\pm$ 0.0057 F1 and Gradient Boosting 0.9469 $\pm$ 0.0054; full model-wise results are exported in `generator_seed_metrics.csv`, `generator_seed_summary.csv`, and `generator_seed_config.json`.

B. Difficulty Progression

Table III compares Random Forest with the tuned four-signal rule across the three difficulty regimes.

For RQ2, performance decreases as the benchmark becomes harder. This difficulty progression is a sanity check: Adaptive attacks are harder because they move toward normal behavior instead of simply increasing request volume. The difficulty levels are not intended to represent measured real-world prevalence. They are controlled stress-test regimes, so absolute scores should be compared within the benchmark rather than interpreted as production estimates.

C. Generator-Shift Robustness

The shifted held-out evaluation tests sensitivity to assumptions encoded by the original synthetic generator. Models are trained on the original Overlap training portion and evaluated on label-preserving shifted versions of the held-out test rows. No model, scaler, selected feature set, model-score threshold, or rule threshold is refitted or retuned on the shifted evaluation data.

The shifted evaluation modifies four controls: more ambiguous benign bursts, reduced attack-volume and failure-rate separation, more distributed IP/device reuse, and altered timing/noise assumptions.

The degradation under generator-shift evaluation indicates dependence on the chosen simulation assumptions. Under this generator-shift evaluation, the learned baselines retain higher F1 scores than the fixed rules.

D. Additional Stress Tests

These stress tests examine whether performance depends on feature count, attack strength, or a single dominant feature group. Random Forest remains close to the full model with Top-15 features (0.9608 F1) but drops with Top-5 features (0.9212 F1). At low attack intensity, Random Forest recall falls to 0.8455 and the tuned rule baseline to 0.7099, suggesting that low-intensity attacks are the most difficult stress case in this benchmark.

Feature-group ablation suggests that the model combines several weak signals rather than relying on one dominant feature group. Removing context is the strongest ablation, but the model still obtains 0.9514 F1.

For RQ3, feature-group ablation indicates that no single group dominates performance. Removing the context/routing group produces the largest decrease, reducing Random Forest F1 from 0.9617 to 0.9514, while permutation importance shows strong model reliance on contextual risk, sequential-pattern, velocity, and infrastructure-reuse features. The representative error-case summary characterizes false-positive and false-negative patterns rather than attributing errors to a single feature group.

On the seed-42 held-out test set, the binary Random Forest detector achieves conditional recall of 0.9709 for flooding, 0.9814 for SMS pumping, and 0.9833 for sequential spray. These values are subtype-conditioned recalls of the binary detector rather than multiclass classification results.

Because success_rate and failure_rate are deterministic complements, they should not be interpreted as independent information sources. The reproduction code includes `complement_identity_sanity.csv` to document the deterministic relationship and `complement_feature_sanity.csv` to compare the full feature set with variants removing success_rate or failure_rate under the same generator seed, split seed, and Random Forest configuration.

This diagnostic does not change the main findings. In the low-cost sanity check, removing success_rate changes F1 from 0.9617 to 0.9623, and removing failure_rate changes F1 to 0.9647. The result indicates redundancy between the two deterministic complements, but this should not be interpreted as evidence that either field is unnecessary in real OTP systems.

E. Threshold Modes and Error Analysis

At threshold 0.3, Random Forest recall is 0.9920 with 95 false positives. At threshold 0.5, it produces 55 false positives and 23 false negatives. At threshold 0.7, precision increases to 0.9699, but recall drops to 0.9003 and false negatives rise to 100. For RQ4, this supports different evaluation modes: lower thresholds are more suitable for warning-oriented evaluation, while higher thresholds are more appropriate for conservative analyst-facing settings.

The rule-baseline recall is not strictly monotonic across attack-intensity levels because the thresholds remain fixed while the intensity transformation changes several correlated features simultaneously.

Random Forest held-out test performance under training-selected cost-sensitive thresholds shows the expected precision/recall tradeoff: FN:FP = 1:1 selects threshold 0.5 (precision 0.9469, recall 0.9771, 55 false positives, 23 false negatives), FN:FP = 5:1 selects 0.3 (0.9128, 0.9920, 95, 8), FN:FP = 10:1 selects 0.2 (0.8720, 0.9980, 147, 2), and FN:FP = 1:5 selects 0.8 (0.9848, 0.8375, 13, 163). These are relative decision weights, not measured SMS cost or business loss. Lower thresholds accept more flagged windows, while higher thresholds miss more attacks; this may support early-warning risk-scoring evaluation, but not standalone automatic-blocking.

A base-rate sensitivity calculation using the seed-42 held-out true-positive and false-positive rates shows that precision depends strongly on assumed attack prevalence. Adjusted precision is 0.0242 at 0.1% prevalence, 0.2004 at 1%, 0.5664 at 5%, and 0.7339 at 10%, compared with 0.9469 at the benchmark prevalence. These adjusted values are mathematical projections from benchmark rates, not deployment estimates, and are exported in `prevalence_sensitivity.csv`. Manuscript values are rounded to four decimals, as recorded in `reporting_precision.json`.

The seed-42 confusion matrix contains 1,342 true negatives, 55 false positives, 23 false negatives, and 980 true positives. False positives resemble legitimate burst or delivery-failure windows: they have higher request counts, high failure rates, and repeated infrastructure. False negatives resemble adaptive low-rate attacks: they show more normal-looking success rates, lower prefix concentration, and less extreme velocity.

Representative simulated error patterns are selected deterministically as the samples closest to the median feature profile of their respective error groups in standardized feature space; metadata are exported in `error_case_selection.json` and `representative_error_cases.csv`. The representative false positive is a normal high-volume/delivery-failure-like window with 33 requests, failure_rate 0.6819, repeat_ip_ratio 0.8485, and prefix_concentration 0.6364. The representative false negative is an adaptive low-rate flooding window with 18 requests, failure_rate 0.6293, prefix_concentration 0.3017, and success_rate 0.3707. These are simulated windows, not real user records or operationally ranked incidents.

F. Interpretability and Sanity Checks

Permutation importance is computed on validation folds within the training portion and aggregated across folds. The held-out test set is not used for feature interpretation or ranking. It ranks risk_country_ratio, sequential_phone_score, repeat_phone_ratio, request_velocity_per_sec, otp_requests, carrier_entropy, device_phone_ratio, and repeat_ip_ratio among the most influential features. The fold-level output is exported in `permutation_importance_cv.csv`. This diagnostic should be interpreted cautiously: permutation importance indicates model reliance in this benchmark, not causal importance in real OTP systems.

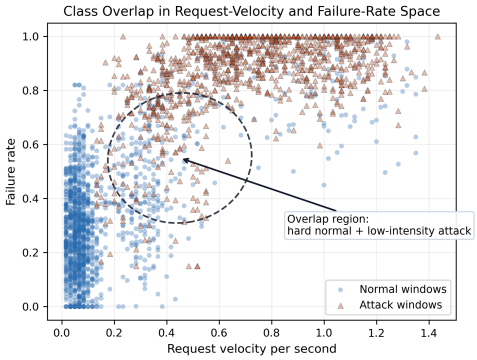

Fig. 2: Class overlap in request-velocity and failure-rate space.

Fig. 2 shows that normal and attack windows overlap in velocity and failure-rate space. The legend distinguishes normal windows from attack windows, and the annotated region highlights hard normal windows and low-intensity attack windows.

The exported feature-correlation check also highlights redundancy between success_rate and failure_rate. These sanity checks support the benchmark design by showing that the simulated classes are not separated by only one obvious feature.

The duplicate audit finds no exact duplicate rows, no cross-label exact duplicate feature vectors, and no exact or rounded train-test feature-vector overlap under the fixed matching rule. This diagnostic reduces one simple leakage risk but does not prove that all near-duplicate or campaign-level leakage is absent.

VII. Discussion

The evaluation emphasizes sensitivity to feature and generator assumptions rather than the absolute Random Forest score. The generator-shift result shows that simulated OTP flooding detection is sensitive to changes in benchmark controls.

The performance gap between the rule baselines and learned models is clearest in the harder settings. In the benchmark, rules detect many obvious attacks, but fixed thresholds lose recall more sharply in Overlap, Adaptive, low-intensity, and generator-shift tests. Lightweight models may add value by combining weak evidence across velocity, verification outcome, infrastructure reuse, destination concentration, and context.

The operating conclusion is deliberately conservative. The evaluation does not justify using any evaluated detector as a standalone automatic-blocking mechanism. The benchmark may support research on analyst-facing early-warning risk-scoring workflows; operational response policies require separate production validation.

The results suggest that request velocity, verification outcome, destination concentration, contextual risk, and infrastructure reuse are candidate signals for future validation. Deep learning is not evaluated because the goal is benchmark analysis rather than model novelty; lightweight tabular models are cheaper and easier to inspect for small window-level datasets.

OTPFloodGuard makes explicit the evidence sources, feature definitions, difficulty controls, failure cases, and parameters that future aggregate data could replace.

Future work can replace simulated parameter ranges with aggregate statistics from a real OTP service, use real benign traffic with injected attack windows, or validate the full pipeline against labeled private incidents.

VIII. Threats to Validity

The primary limitation is synthetic data. OTPFloodGuard cannot fully represent production traffic, carrier routing, user demographics, regional behavior, or adaptive attacker strategy. Its labels come from the generator, not incident investigation. Public evidence constrains the risk signals but does not provide exact distributions or labeled request logs. The benchmark uses a deliberately elevated attack proportion to support controlled model comparison and error analysis; this should not be interpreted as an operational attack base rate, and precision, alert volume, and analyst workload may differ substantially when attacks are rare. The evaluation also uses random row-level splits of simulated windows and does not test temporal, campaign-level, or source-grouped generalization. The seven-split analysis measures partition sensitivity within one generated benchmark, not independent dataset replication. The five-seed generator analysis reduces dependence on a single synthetic instance but does not establish real-world distributional validity.

The generator-shift evaluation is a controlled, class-conditional, label-preserving stress test rather than an observed production distribution shift. Probability calibration is not evaluated; the reported thresholds are decision cutoffs selected from training-set predictions rather than calibrated estimates of attack likelihood. Some outcome features, such as success_rate and failure_rate, may also be delayed in practical early-warning settings. Finally, the Adaptive blend and attack-intensity settings are simplified approximations rather than measured attacker ratios, and the study does not evaluate live rate limiting, analyst workload, drift monitoring, user friction, privacy review, or automatic blocking. Any future production validation must use anonymized or aggregate OTP logs under institutional privacy requirements.

IX. Conclusion

OTPFloodGuard is a public-evidence-constrained benchmark for studying lightweight OTP flooding detection under simulated conditions. It does not show that any model is ready for real authentication systems. Instead, it provides a transparent and reproducible way to connect public OTP/SMS abuse evidence to threat assumptions, features, difficulty levels, baseline models, and failure analysis. The results suggest that short-window behavioral features merit further study for early-warning risk-scoring evaluation. Each assumption and parameter can be replaced when aggregate production statistics become available. Real-world validation on anonymized production logs remains necessary before operational deployment.

References

[1] Twilio, "What is SMS pumping fraud?" Twilio Docs, n.d. [Online]. Available: https://www.twilio.com/docs/glossary/what-is-sms-pumping-fraud.

[2] M. Piccirilli, "Reduce OTP fraud with Twilio Verify's fraud detection," Twilio Blog, 14 Jul. 2022. [Online]. Available: https://www.twilio.com/en-us/blog/developers/best-practices/verify-otp-fraud-detection.

[3] IPQualityScore, "SMS pumping detection and toll fraud prevention," n.d. [Online]. Available: https://www.ipqualityscore.com/toll-fraud-prevention/sms-pumping-detection.

[4] RingCaptcha, "Voice and SMS OTP resource exhaustion attacks," n.d. [Online]. Available: https://www.ringcaptcha.com/voice-sms-otp-resource-exhaustion-attacks.

[5] MITRE ATT&CK, "Resource Hijacking: SMS Pumping, T1496.003," version 1.0, last modified 15 Apr. 2025. [Online]. Available: https://attack.mitre.org/techniques/T1496/003/.

[6] Prelude, "2025 SMS Pumping Fraud Report: What 205 Million Authentication Requests Revealed," 1 Jun. 2026. [Online]. Available: https://prelude.so/blog/2025-sms-fraud-report.

[7] M. E. Irarrazaval, S. Maldonado, J. E. Perez, and C. M. Vairetti, "Telecom traffic pumping analytics via explainable data science," Decision Support Systems, vol. 150, 113559, 2021.

[8] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, and N. Choi, "Preventing Artificially Inflated SMS Attacks through Large-Scale Traffic Inspection," in Proc. 34th USENIX Security Symp. (USENIX Security 25), pp. 5405-5423, 2025. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/huh.

[9] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in Proc. Int. Conf. Information Systems Security and Privacy (ICISSP), pp. 108-116, 2018.

[10] T. A. Almeida, J. M. G. Hidalgo, and A. Yamakami, "Contributions to the study of SMS spam filtering: new collection and results," in Proc. ACM Symp. Document Engineering, pp. 259-262, 2011.

[11] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Learned lessons in credit card fraud detection from a practitioner perspective," Expert Systems with Applications, vol. 41, no. 10, pp. 4915-4928, 2014.

[12] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: a survey," ACM Computing Surveys, vol. 41, no. 3, Article 15, 2009.

[13] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5-32, 2001.

[14] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.

[15] A. Altmann, L. Tolosi, O. Sander, and T. Lengauer, "Permutation importance: a corrected feature importance measure," Bioinformatics, vol. 26, no. 10, pp. 1340-1347, 2010.

[16] Z. Lei, Y. Nan, Y. Fratantonio, and A. Bianchi, "On the insecurity of SMS one-time password messages against local attackers in modern mobile devices," in Proc. Network and Distributed System Security Symp., 2021.

[17] F. Pedregosa et al., "Scikit-learn: machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[18] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLOS ONE, vol. 10, no. 3, e0118432, 2015.

[19] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symp. Security and Privacy, pp. 305-316, 2010.

[20] A. Paullada, I. D. Raji, E. M. Bender, E. Denton, and A. Hanna, "Data and its (dis)contents: A survey of dataset development and use in machine learning research," Patterns, vol. 2, no. 11, 100336, 2021.

[21] V. Borisov, T. Leemann, K. Sessler, J. Haug, M. Pawelczyk, and G. Kasneci, "Deep neural networks and tabular data: A survey," IEEE Transactions on Neural Networks and Learning Systems, vol. 35, no. 6, pp. 7499-7519, 2024.